

This is a **good systems/OS project**, and it can be made substantially stronger than simply “compare pipes, sockets, and queues.” The interesting research problem is:

Can an IPC library dynamically choose the most suitable Linux IPC transport for a workload, using observed message size, message rate, queue depth, and latency/throughput measurements?

That gives you both an implementation project and an experimental/research component.

1. First, refine the project statement

Your current statement is:

Adaptive IPC Framework for Linux

Pipes, sockets and queues have different performance characteristics.
Automatically select the IPC mechanism based on message size/frequency.

I would change it to:

Adaptive IPC Framework for Linux: Runtime Selection of IPC Mechanisms Based on Workload Characteristics

Problem

Linux provides several IPC mechanisms, including pipes, UNIX domain sockets, and POSIX message queues. These mechanisms have different semantics, buffering characteristics, message-boundary behavior, resource limits, and performance under different workloads.

For example, Linux pipes are byte streams and have finite capacity; writes can block when the pipe is full. Linux also provides configurable pipe capacity through `F_SETPIPE_SZ`. man7.org

UNIX-domain sockets provide local IPC and support stream, datagram, and sequenced-packet communication. They can also preserve message boundaries when using appropriate socket types. man7.org

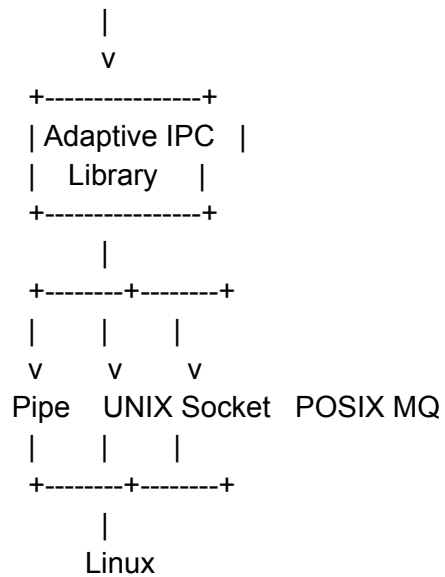
POSIX message queues are explicitly message-oriented, support priorities, have configurable queue/message limits, and are implemented through dedicated kernel interfaces. man7.org+1

Therefore, **there isn't necessarily one IPC mechanism that is optimal for every workload.**

Proposed solution

Build a common IPC abstraction:

Application



The application should not need to know which mechanism is being used.

For example:

```
ipc_send(channel, data, size);
```

instead of:

```
write(pipe_fd, data, size);
```

or:

```
send(socket_fd, data, size, 0);
```

or:

```
mq_send(mq, data, size, priority);
```

Your framework decides which transport to use.

2. The important correction: don't make "message frequency" your only input

This is probably the most important design point.

You initially proposed:

select IPC mechanism based on message size/frequency

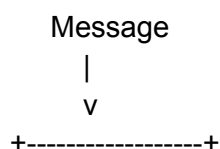
That's a good starting point, but for a serious project I would use:

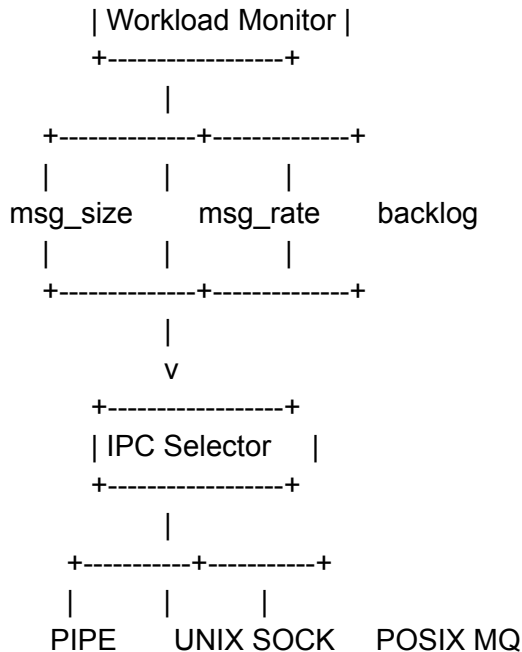
message size
message rate
queue/backlog depth
latency requirement
concurrency
current IPC performance

For example:

Workload	Potentially important factor
64-byte messages $\times$ 100/s	latency + syscall overhead
64-byte messages $\times$ 100,000/s	syscall rate + batching
4 KB $\times$ 1,000/s	throughput
1 MB $\times$ 10/s	bulk-transfer efficiency
Bursty workload	buffering/backpressure
Multiple producers	contention + message semantics
Multiple consumers	scheduling + queue semantics

So your adaptive algorithm should eventually look like:





3. What is the actual limitation?

You should be precise here.

It would be **incorrect** to write:

"Linux documentation says pipes are faster for small messages and sockets are faster for large messages."

The Linux man pages generally document **semantics and implementation limits**, not universal performance rankings.

Instead, you should establish the limitation from two sources:

A. Linux documentation establishes that the mechanisms are fundamentally different

Pipes

The Linux [pipe\(7\)](#) documentation says pipes are **byte streams**, not message-oriented channels. They have finite capacity, and when full, writes can block or fail depending on [O_NONBLOCK](#). Linux also documents [PIPE_BUF](#) atomicity and configurable pipe capacity. [man7.org](#)

Linux pipe(7) — official Linux man-pages documentation

This is important for your project because a pipe's behavior depends on:

message size
pipe capacity
reader speed
writer speed
blocking/nonblocking mode
number of writers

B. UNIX domain sockets have different semantics

Linux's [unix\(7\)](#) documentation describes UNIX-domain sockets as a local IPC mechanism and documents three important socket types:

SOCK_STREAM
SOCK_DGRAM
SOCK_SEQPACKET

[SOCK_DGRAM](#) preserves message boundaries, while [SOCK_SEQPACKET](#) also provides connection-oriented, ordered, message-boundary-preserving communication. [man7.org](#)

Linux unix(7) — official Linux man-pages documentation

You can also use [socketpair\(AF_UNIX, ...\)](#) for directly connected local processes. [man7.org](#)

Linux socketpair(2) — official Linux man-pages documentation

C. POSIX message queues are fundamentally different again

Linux's [mq_overview\(7\)](#) explicitly describes POSIX message queues as a mechanism for exchanging **messages**.

They provide:

- message boundaries

- priorities
- configurable maximum message size
- configurable queue capacity
- blocking/nonblocking operations
- asynchronous notification

Linux documents a default `msgsize_max` of 8192 bytes, with kernel limits depending on the kernel version/configuration. man7.org+1

Linux mq_overview(7) — official Linux man-pages documentation

This is actually a strong argument for your project:

Pipe:

- byte stream
- finite buffer
- PIPE_BUF semantics

UNIX socket:

- stream/datagram/seqpacket
- socket buffering
- message boundaries depending on type

POSIX MQ:

- message-oriented
- priorities
- queue limits
- maximum message size

These aren't simply three implementations of the same thing.

4. There is actual research supporting your premise

This is where your project becomes much more academically defensible.

A peer-reviewed study titled “**Performance analysis of five interprocess communication mechanisms across UNIX operating systems**” evaluated pipes, FIFOs, message queues, shared memory, and UNIX-domain sockets using producer/consumer benchmarks.

The study found that performance differences were associated with factors including:

- memory allocation
- transfer-buffer sizes
- data-transfer mechanisms
- locking
- implementation complexity

It also emphasizes that IPC performance can vary significantly with implementation and workload. ScienceDirect

Performance analysis of five interprocess communication mechanisms across UNIX operating systems — Journal of Systems and Software

That's a **much better academic reference** than making an unsupported statement like “pipes are faster than sockets.”

There is also a more recent Linux-focused comparison of anonymous pipes, named pipes, UNIX sockets, and TCP sockets that explicitly observes that performance changes with block/message size. In its tested environment, pipes performed better at smaller blocks while UNIX sockets became substantially faster at larger blocks. Importantly, those are **experimental results, not universal Linux guarantees**. Baeldung on Kotlin

IPC Performance Comparison: Anonymous Pipes, Named Pipes, Unix Sockets, and TCP Sockets

That distinction should appear in your report.

5. Your project should have TWO major components

Don't make this merely:

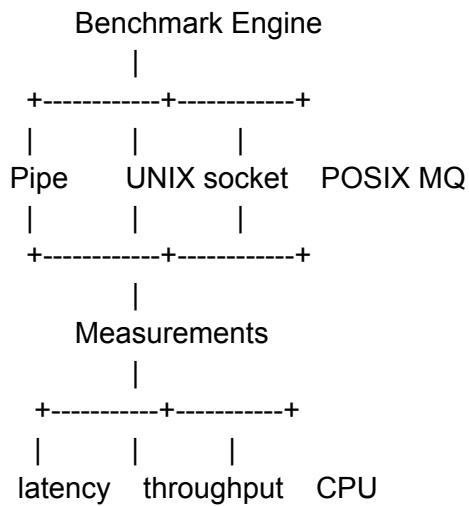
```
if (size < 1000)
    use pipe;
else
    use socket;
```

That would be too simplistic.

Instead:

Component 1 — IPC Benchmark Engine

First determine the performance characteristics experimentally.



Measure:

Latency

Time required for:

send → receive

Use:

```
clock_gettime(CLOCK_MONOTONIC_RAW, ...);
```

Preferably measure **round-trip latency** initially:

Process A

|

| request

v

Process B

|

| response

v

Process A

Then:

$\text{latency} = (\text{end} - \text{start}) / 2$

for an approximate one-way latency when appropriate.

Throughput

Measure:

bytes transferred / second

and:

messages / second

These are different metrics.

For example:

IPC A:

1 GB/s

100,000 msg/s

IPC B:

800 MB/s

500,000 msg/s

Which is better depends on the application.

CPU consumption

Measure:

CPU cycles/message

CPU %

context switches

system calls

You can use Linux tools such as:

perf stat

perf record

and inspect:

cycles

instructions

context-switches

cpu-migrations

cache-misses

page-faults

6. Create a proper benchmark matrix

This is probably the most important experimental part of the project.

Test several message sizes:

8 B

64 B

128 B

256 B

512 B

1 KB

2 KB

4 KB
8 KB
16 KB
32 KB
64 KB
128 KB
256 KB
1 MB

Don't stop at just:

small
medium
large

because you want to discover the **crossover points**.

Then test different message rates:

10 msg/s
100 msg/s
1,000 msg/s
10,000 msg/s
100,000 msg/s
1,000,000 msg/s

Then produce something like:



Your actual graph will come from your experiments.

7. Don't hard-code the result from someone else's benchmark

This is crucial.

You shouldn't say:

"UNIX socket is always best for large messages."

Instead:

"On our test platform, UNIX-domain sockets achieved the best throughput above X KB under workload Y."

Then:

"The adaptive framework learns or uses these measured characteristics to select the transport."

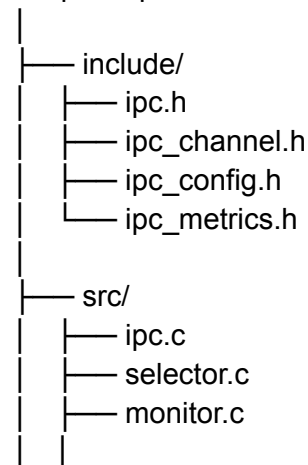
That makes your project scientifically much stronger.

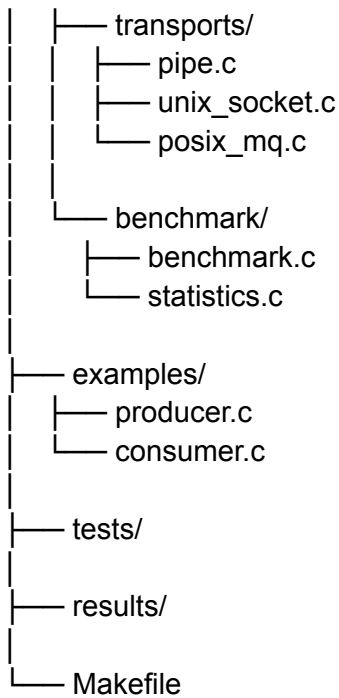
8. Proposed framework architecture

I'd implement the framework in C.

Something like:

adaptive-ipc/





9. Define a common IPC interface

For example:

```
typedef enum {  
    IPC_PIPE,  
    IPC_UNIX_SOCKET,  
    IPC_POSIX_MQ  
} ipc_transport_t;
```

Then:

```
typedef struct {  
    ipc_transport_t transport;  
  
    size_t message_size;  
  
    double message_rate;  
  
    double latency_requirement;
```

```
    int priority;  
} ipc_config_t;
```

Your common interface could be approximately:

```
int ipc_create(ipc_config_t *config);  
  
int ipc_send(  
    ipc_channel_t *channel,  
    const void *data,  
    size_t size  
);  
  
int ipc_receive(  
    ipc_channel_t *channel,  
    void *buffer,  
    size_t buffer_size  
);  
  
int ipc_close(ipc_channel_t *channel);
```

The application sees only:

```
ipc_send(channel, data, size);
```

The framework internally chooses:

```
pipe  
socket  
message queue
```

10. Transport abstraction

Use function pointers / virtual interface style.

For example:

```
struct ipc_transport_ops {  
    int (*create)(ipc_channel_t *);
```

```

int (*send)(ipc_channel_t *, const void *, size_t);
int (*receive)(ipc_channel_t *, void *, size_t);
int (*close)(ipc_channel_t *);
};

```

Then:

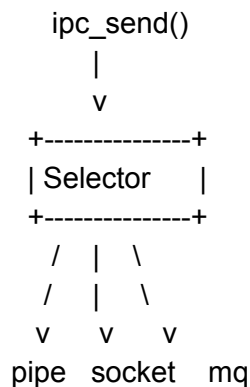
```

pipe_ops
socket_ops
mq_ops

```

all implement the same interface.

This gives you:



11. The selector is the core of your project

Initially, implement a **static selector**.

For example:

```

if message_size <= X
    choose PIPE

else if message_size <= Y
    choose POSIX_MQ

else
    choose UNIX_SOCKET

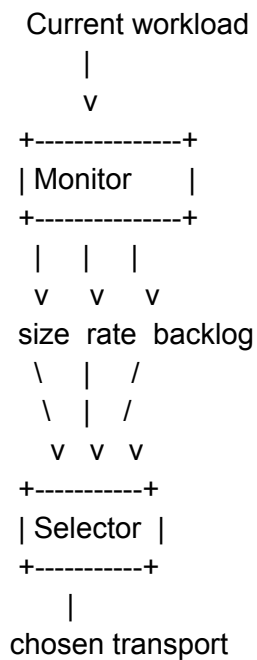
```

But don't decide X and Y yourself.

Your benchmark should determine them.

12. Then make it adaptive

The more interesting version is:



Maintain an exponentially weighted moving average:

EWMA_latency
EWMA_throughput
EWMA_CPU

For each transport.

For example:

new_average =
alpha * current +

$(1 - \alpha) * \text{old_average};$

where:

$\alpha = 0.1$

might be a starting point.

13. Use a cost function instead of simple if-statements

This would make the project considerably better.

For every available IPC mechanism calculate:

Score =
 $w_1 * \text{normalized_latency}$
 $+ w_2 * \text{normalized_cpu}$
 $+ w_3 * \text{normalized_throughput_cost}$
 $+ w_4 * \text{queue_pressure}$

Then choose:

$\text{transport} = \text{argmin}(\text{score})$

For example:

	Pipe	Socket	MQ
Latency	2.1	3.0	2.5
CPU	1.2	1.5	2.0
Throughput	2.0	1.0	1.8
Queue pressure	3.0	1.0	2.0

Score	2.2	1.4	2.0

↓

UNIX socket

The weights could depend on the application:

latency-sensitive:

latency weight = high

bulk transfer:

throughput weight = high

CPU constrained:

CPU weight = high

14. But there is a major problem with runtime switching

This is an important issue that will impress your evaluator if you identify it yourself.

Suppose you're currently using:

PIPE

and your selector decides:

switch to UNIX SOCKET

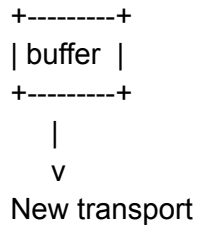
What happens to messages already inside the pipe?

You cannot simply close the pipe.

You need a migration protocol.

For example:

```
Old transport
|
| drain
v
```



One approach:

1. Stop new messages temporarily.
2. Drain old transport.
3. Create new transport.
4. Transfer/drain remaining messages.
5. Switch producer/consumer.
6. Destroy old transport.

But this introduces a **switching cost**.

Therefore your selector should have a hysteresis rule:

Don't switch unless:

$\text{new_score} + \text{switching_cost} < \text{current_score}$

or:

switch only if new transport
is at least 20% better
for N consecutive measurements

This prevents:

PIPE
SOCKET
PIPE
SOCKET
PIPE
SOCKET

oscillation.

15. Even better: don't switch too frequently

Use a decision window.

For example:

monitor every 100 ms

but

allow transport changes only every 1 second

And:

minimum improvement = 15%

So:

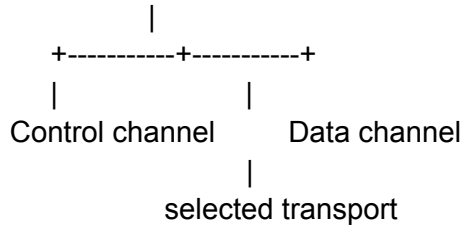
```
if (new_score < old_score * 0.85 &&  
    time_since_last_switch > MIN_SWITCH_INTERVAL)  
{  
    switch_transport();  
}
```

The exact values should be experimentally evaluated.

16. An even more interesting research idea

Instead of completely switching transports, you can have:

Adaptive IPC



For example:

UNIX socket
+
control messages
+
adaptive monitoring

The control channel tells the processes:

SWITCH_TO_PIPE

Then both processes migrate consistently.

17. What should the benchmark actually test?

Create a test matrix.

Dimension 1 — IPC mechanism

Pipe
UNIX SOCK_STREAM
UNIX SOCK_DGRAM / SOCK_SEQPACKET
POSIX message queue

I'd initially limit the implementation to:

pipe
UNIX SOCK_STREAM
POSIX MQ

Then optionally add:

UNIX SOCK_SEQPACKET

later.

Dimension 2 — message size

64 B
128 B
256 B
512 B
1 KB
2 KB
4 KB
8 KB
16 KB
32 KB
64 KB
128 KB
256 KB
1 MB

Dimension 3 — message frequency

100/s
1K/s
10K/s
100K/s
1M/s

Dimension 4 — workload

constant
bursty
producer faster than consumer
consumer faster than producer

Dimension 5 — number of processes

Start with:

1 producer
1 consumer

Then:

2 producers
1 consumer

and:

1 producer
2 consumers

if your design supports it.

18. Your benchmark results should look like this

Don't invent numbers.

Generate them experimentally.

For example:

Message Size: 64 B

Transport	Latency	Throughput	CPU
Pipe	measured	measured	measured
Unix socket	measured	measured	measured
POSIX MQ	measured	measured	measured

Then:

Message Size: 4 KB

Transport	Latency	Throughput	CPU

Pipe	measured	measured	measured
Unix socket	measured	measured	measured
POSIX MQ	measured	measured	measured

Then:

Message Size: 1 MB

Transport	Latency	Throughput	CPU

Pipe	measured	measured	measured
Unix socket	measured	measured	measured
POSIX MQ	measured	N/A	measured

The last point is particularly interesting because POSIX message queues have configurable maximum message sizes. Linux documents [msgsize_max](#) and related limits. [man7.org](#)

19. Your framework can use two phases

I strongly recommend this architecture.

Phase 1 — Calibration

When the framework starts:

Run short benchmark

|
v

Measure each IPC

|
v

Create performance model

|
v

Store thresholds

For example:

IPC_PROFILE:

PIPE:

optimal message size: ...

latency: ...

throughput: ...

UNIX_SOCKET:

optimal message size: ...

latency: ...

throughput: ...

POSIX_MQ:

optimal message size: ...

latency: ...

throughput: ...

Phase 2 — Runtime adaptation

Then:

Application

|

v

send(message)

|

v

monitor workload

|

v

calculate IPC scores

|

v

choose transport

This is much better than assuming that benchmarks from another computer apply to your current machine.

20. A potentially better approach: online learning

If you want to make the project more advanced, the selector can learn from actual execution.

Initially:

unknown

Try each transport on suitable workloads.

Then record:

(message_size,
message_rate,
transport,
latency,
throughput,
CPU)

Build a simple model.

You don't need deep learning.

A simple regression or lookup table is enough:

size range × rate range → best IPC

For example:

	Message rate		
	low	medium	high
< 1 KB	pipe	pipe	MQ
1-8 KB	pipe	socket	socket
> 8 KB	socket	socket	socket

Those entries are examples only; your experiments must determine the actual table.

21. Important limitation of your proposed project

You should explicitly state:

There is no universally optimal IPC mechanism.

Performance depends on:

- kernel version
- CPU architecture
- CPU cache
- scheduler
- message size
- message rate
- buffer sizes
- producer/consumer behavior
- number of processes
- system load
- IPC configuration

The academic literature supports the broader observation that IPC performance depends on implementation details such as buffer sizes, memory allocation, transfer mechanisms, and locking. ScienceDirect

Therefore your project shouldn't claim:

"Our algorithm determines the universally fastest IPC."

Instead:

"Our algorithm dynamically selects the IPC mechanism expected to minimize a specified cost function under the current workload and system conditions."

That's a much stronger research claim.

22. Recommended project objectives

You could put these directly into your proposal.

Objective 1

Implement a unified IPC API supporting:

- Pipe
- UNIX domain socket
- POSIX message queue

Objective 2

Develop a benchmarking framework to measure:

- latency
- throughput
- messages/sec
- CPU utilization
- context switches

Objective 3

Characterize the performance of each IPC mechanism under different:

- message sizes
- message rates
- workloads

Objective 4

Develop a workload-aware IPC selection algorithm.

Objective 5

Implement runtime monitoring and adaptive transport selection.

Objective 6

Evaluate the adaptive framework against:

- always-pipe
- always-socket

always-message-queue

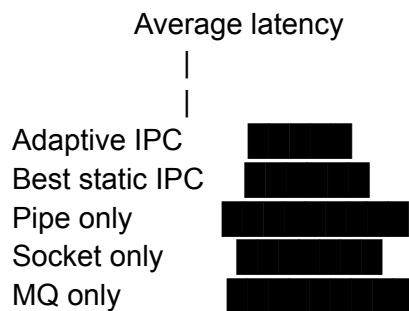
This last comparison is **very important**.

23. Your final evaluation should answer this question

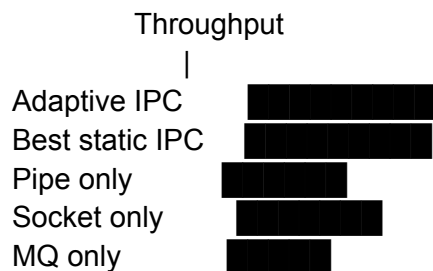
The final experiment shouldn't merely show:

Pipe = X
Socket = Y
MQ = Z

It should show:



and:



Your key metric could be:

Adaptation gain

adaptation_gain =
adaptive_performance / best_static_performance

For latency, invert appropriately:

latency_improvement =
(static_latency - adaptive_latency)
/ static_latency × 100

Then you can make a meaningful conclusion.

24. Suggested implementation stages

I would do the project in this order.

Stage 1 — Individual transports

Implement:

pipe.c
unix_socket.c
posix_mq.c

and verify correctness.

Stage 2 — Common API

Create:

ipc_create()
ipc_send()
ipc_receive()
ipc_close()

Stage 3 — Benchmark framework

Automate:

message sizes
message rates
iterations
latency
throughput
CPU

Output:

transport,message_size,message_rate,latency,throughput,cpu
pipe,64,1000,...
socket,64,1000,...
mq,64,1000,...

Stage 4 — Static selector

Use experimentally derived thresholds.

Stage 5 — Runtime monitoring

Collect:

EWMA latency
EWMA throughput
message rate
queue depth

Stage 6 — Adaptive selector

Implement:

score()
argmin()
hysteresis
switching cost

Stage 7 — Runtime migration

Implement safe switching between transports.

Stage 8 — Evaluation

Compare:

Adaptive

Pipe only

Socket only

MQ only

Oracle/best-static

25. Documentation you should cite in your project report

I would use these as your primary references.

Official Linux documentation

1. Linux **pipe(7)**

pipe(7) — Linux manual page

Use it to support:

- pipes are byte streams
- finite pipe capacity
- blocking behavior
- **PIPE_BUF**
- atomicity
- configurable pipe size

man7.org

2. Linux **unix(7)**

unix(7) — Linux manual page

Use it to support:

- UNIX-domain IPC
- `SOCK_STREAM`
- `SOCK_DGRAM`
- `SOCK_SEQPACKET`
- message-boundary differences

man7.org

3. Linux `mq_overview(7)`

mq_overview(7) — Linux manual page

Use it to support:

- POSIX message queues
- message semantics
- priorities
- queue limits
- message-size limits
- notification

man7.org+1

4. Linux `socketpair(2)`

socketpair(2) — Linux manual page

Useful for your implementation of connected local UNIX-domain socket IPC. man7.org

Academic reference

Performance analysis of five interprocess communication mechanisms across UNIX operating systems, Journal of Systems and Software.

Academic IPC performance study

This is particularly valuable because it experimentally studies different IPC mechanisms and explains why their performance differs. ScienceDirect

26. A polished problem statement for your proposal

You can use this as the basis of your proposal during design, which may result in suboptimal performance when workload characteristics change. Previous experimental research has demonstrated that IPC performance can vary according to factors such as buffer size, transfer mechanism, memory management, and implementation details. actual proposal:

Adaptive IPC Framework for Linux

Linux provides multiple inter-process communication mechanisms, including pipes, UNIX-domain sockets, and POSIX message queues. Although these mechanisms provide similar high-level functionality, their underlying semantics, buffering behavior, message handling, resource limits, and performance characteristics differ. Linux pipes operate as byte streams with finite buffering, UNIX-domain sockets support different communication models such as streams and sequenced packets, and POSIX message queues provide message-oriented communication with priorities and configurable message and queue limits. man7.org+2

Existing applications generally select an IPC mechanism statically during design, which may result in suboptimal performance when workload characteristics change. Previous experimental research has demonstrated that IPC performance can vary according to factors such as buffer size, transfer mechanism, memory management, and implementation details. ScienceDirect

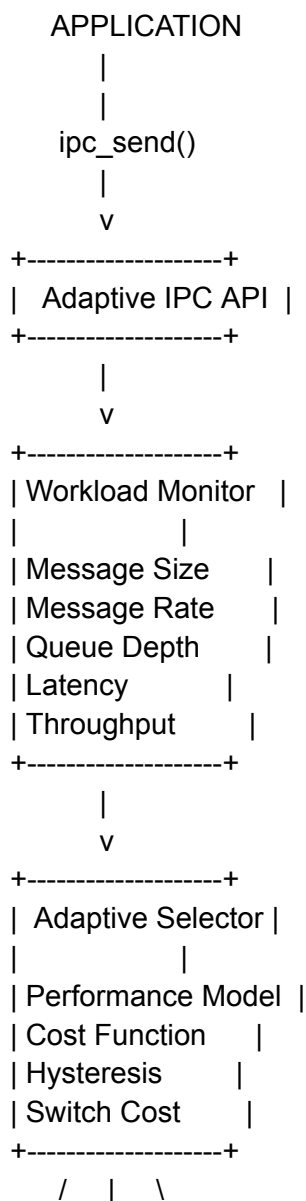
This project proposes an adaptive IPC framework for Linux that provides a common IPC abstraction while dynamically selecting an appropriate underlying IPC mechanism according to workload characteristics such as message size, message rate, queue pressure, latency, and throughput. The framework will benchmark the supported mechanisms, construct a performance model, monitor runtime workload

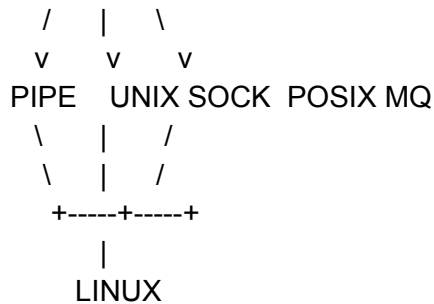
characteristics, and dynamically select or migrate to a more appropriate IPC transport.

The framework will be evaluated against fixed IPC strategies using latency, throughput, CPU utilization, message rate, and adaptation overhead as evaluation metrics.

27. our recommended final project architecture

If this is a **final-year/semester project**, I'd settle on this:



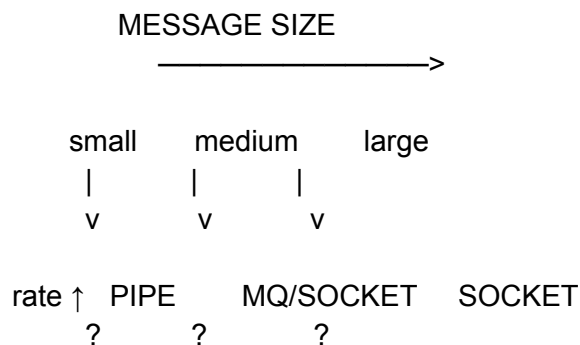


The research question becomes

Can runtime workload-aware selection of Linux IPC mechanisms reduce communication latency/CPU cost or increase throughput compared with a statically selected IPC mechanism?

That's a **much stronger project** than simply “implement pipes, sockets and queues.”

And the most important experimental result will be your **crossover map**:



where the ? values are **discovered experimentally on your Linux system**, rather than assumed beforehand.

If you implement the project this way, you have a clear combination of **Linux systems programming + benchmarking + adaptive algorithm + runtime systems**, with official Linux documentation and prior IPC-performance research supporting the motivation.